

난독화 된 한글 리뷰 복원 AI 프로젝트

B팀 박수빈, 박소현, 신윤서

1. 데이터 이해

1) 난독화 리뷰의 특징

본 과제에서 다루는 데이터는 한글 리뷰를 난독화한 텍스트로, 원문을 알아보기 어렵게 변형한 데이터이다. 분석 결과, 난독화는 주로 한글 모음을 다른 모음으로 교체하는 방식으로 이루어진다. 예를 들어 '눈'이 '눅'으로, '요'가 '오'로, '다'가 '다'나 '따'로 바뀌는 패턴이 빈번하게 관찰되었다. 이외에도 초성을 남기는 방식(ㅋ, ㅎ, ㅍ, ...), 특수문자 삽입, 숫자 및 영문자 혼용 등의 패턴이 확인되었다.

주목할 점은 입력(난독화)과 출력(원문)의 문자 수가 97.1%의 경우에서 동일하다는 것이다. 이는 난독화가 문자를 추가하거나 삭제하는 방식이 아니라, 기존 문자를 다른 문자로 교체하는 방식으로 주로 이루어짐을 의미한다.

```
print("=== train 결측치 ===")
print(train.isnull().sum())

print("\n=== test 결측치 ===")
print(test.isnull().sum())

print("\n=== train 데이터 타입 ===")
print(train.dtypes)
```

```
=== train 결측치 ===
ID      0
input    0
output    0
dtype: int64
```

```
=== test 결측치 ===
ID      0
input    0
dtype: int64
```

```
=== train 데이터 타입 ===
ID      object
input    object
output    object
dtype: object
```

2) 주요 난독화 패턴 빈도

```
import re

patterns = {
    '초성 포함 (ㄱ-ㅎ)': 0,
    '특수문자 포함': 0,
    '숫자 포함': 0,
    '영문자 포함': 0,
}

for inp in train['input']:
    inp = str(inp)
    if re.search(r'[ㄱ-ㅎ]', inp):
        patterns['초성 포함 (ㄱ-ㅎ)'] += 1
    if re.search(r'[@#$%^&*()\.\,\'\"?~\-\_]', inp):
        patterns['특수문자 포함'] += 1
    if re.search(r'\d', inp):
        patterns['숫자 포함'] += 1
    if re.search(r'[a-zA-Z]', inp):
        patterns['영문자 포함'] += 1

print("=== 난독화 패턴별 빈도 ===")
for k, v in patterns.items():
    print(f" {k}: {v}건 ({v/len(train)*100:.1f}%)")
```

```
=== 난독화 패턴별 빈도 ===
초성 포함 (ㄱ-ㅎ): 1077건 (9.6%)
특수문자 포함: 10687건 (94.9%)
숫자 포함: 2848건 (25.3%)
영문자 포함: 466건 (4.1%)
```

```

) from collections import Counter

diff_pairs = []

for _, row in train.iterrows():
    inp = str(row['input'])
    out = str(row['output'])
    if len(inp) == len(out):
        for a, b in zip(inp, out):
            if a != b:
                diff_pairs.append((a, b))

counter = Counter(diff_pairs)
top30 = counter.most_common(30)

chars_obf = [pair[0] for pair, _ in top30]
chars_orig = [pair[1] for pair, _ in top30]
counts = [cnt for _, cnt in top30]

print("=== 자주 바뀌는 문자 쌍 TOP 30 (난독화 → 원문) ===")
print(f"{'난독화':^6} {'원문':^6} {'빈도':^6}")
print("-" * 22)
for (obf, orig), cnt in top30:
    print(f"{'obf':^6} → {'orig':^6} {'cnt':^6}회")

```

=== 자주 바뀌는 문자 쌍 TOP 30 (난독화 → 원문) ===		
난독화	원문	빈도
눈	→ 는	8132회
오	→ 요	5541회
늪	→ 니	5491회
다	→ 다	5037회
교	→ 고	4947회
위	→ 이	4935회
도	→ 도	3681회
하	→ 하	3655회
학	→ 하	3257회
많	→ 만	3149회
따	→ 다	3041회
타	→ 다	3035회
써	→ 서	3027회
꼬	→ 고	2983회

3) train/test 차이 + 평균 문장 길이

train데이터에는 난독화된 입력(input)과 정답 원문(output)이 함께 제공되지만, test 데이터에서는 output이 존재하지 않아 모델이 직접 복원 결과를 예측해야 한다. 또한, test 데이터의 평균 문장 길이(168자)가 train(93자)보다 길어, 모델이 긴 문장에 대한 일반화 성능을 갖추어야 했다.

```

train['input_len'] = train['input'].str.len()
train['output_len'] = train['output'].str.len()
test['input_len'] = test['input'].str.len()

print("=== train input 길이 ===")
print(train['input_len'].describe())

print("\n=== train output 길이 ===")
print(train['output_len'].describe())

print("\n=== test input 길이 ===")
print(test['input_len'].describe())

# 입출력 길이 차이
train['len_diff'] = train['input_len'] - train['output_len']
print(f"\n입출력 길이 동일한 비율: {(train['len_diff'] == 0).sum() / len(train) * 100:.1f}%")
print(f"input이 더 긴 비율: {(train['len_diff'] > 0).sum() / len(train) * 100:.1f}%")
print(f"output이 더 긴 비율: {(train['len_diff'] < 0).sum() / len(train) * 100:.1f}%")

```

```

=== train input 길이 ===
count      11263.000000
mean         93.089497
std          98.777700
min           1.000000
25%          33.000000
50%          63.000000
75%         116.000000
max         1381.000000
Name: input_len, dtype: float64

```

```

=== train output 길이 ===
count      11263.000000
mean         93.118707
std          98.766055
min           2.000000
25%          33.000000
50%          63.000000
75%         116.000000
max         1381.000000
Name: output_len, dtype: float64

```

```

=== test input 길이 ===
count      1689.000000
mean        168.101243
std         232.969586
min           6.000000
25%          41.000000
50%          83.000000
75%         176.000000
max         1965.000000
Name: input_len, dtype: float64

```

```

입출력 길이 동일한 비율: 97.1%
input이 더 긴 비율: 0.0%
output이 더 긴 비율: 2.9%

```

4) 문자 단위 F1 Score 평가 방식

본 과제의 평가 지표는 문자 단위 F1 Score이다.

예측된 복원 텍스트와 정답 텍스트를 문자(음절) 단위로 비교하여 Precision과 Recall을 계산하고, 두 값의 조화 평균을 F1 Score로 산출한다.

- **Precision** = 예측 텍스트 중 정답과 일치하는 문자 수 / 예측 텍스트 전체 문자 수
- **Recall** = 정답 텍스트 중 예측과 일치하는 문자 수 / 정답 텍스트 전체 문자 수
- **F1 Score** = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

완벽히 복원한 경우 $F1 = 1.0$, 전혀 복원하지 못한 경우 $F1 = 0.0$ 에 가까워진다.

2. 전체 실행 흐름 설명

1) 데이터 불러오기

- train.csv : 난독화된 입력과 정답 원문 쌍 11,263개
- test.csv : 난독화된 입력만 존재, 1,689개
- sample_submission.csv : 제출 형식 확인용

결측치가 없음을 확인하여 별도의 데이터 정제 없이 바로 활용 가능함을 확인하였다.

2) 텍스트 전처리

모델 학습에 앞서 train데이터를 학습용(train_split)과 검증용(val_split)으로 분리한다. 9:1 비율로 분리하였으며, 이는 모델이 학습 데이터에 과적합되지 않았는지 확인하고 성능을 객관적으로 평가하기 위해 필요한 과정이다.

- train_split : 10,136개 (학습에 사용)
- val_split : 1,127개 (검증에 사용)

3) Dictionary 구축

train_split 데이터에서 난독화된 단어(input)와 원문 단어(output)를 단어 단위로 짝지어

사전(match_dict)을 구축한다. 이 사전은 이후 치환 기반 복원의 핵심 자료로 활용된다. train 데이터에서만 사전을 구축하고 val/test 데이터는 검증 및 예측에만 사용함으로써, 모델이 학습 데이터를 단순 암기하지 않고 일반화 성능을 갖추었는지 평가할 수 있다.

4) Tokenizer 구성

LLM 기반 복원 모델에서는 텍스트를 모델이 이해할 수 있는 숫자(토큰) 형태로 변환하는 Tokenizer가

필요하다. beomi/gemma-ko-2b 모델에 맞는 Tokenizer를 불러와 적용하며, 최대

길이(max_length=256)를 초과하는 문장은 잘라내어 처리한다. Tokenizer 없이는 텍스트 데이터를 모델에 입력할 수 없으므로 필수적인 단계이다.

5) Dataset 구성

pandas DataFrame 형태의 train/val 데이터를 HuggingFace의 Dataset 객체로 변환한다.

SFTTrainer는 HuggingFace Dataset 형식을 기반으로 동작하므로, 이 변환 과정이 없으면 모델 학습 자체가 불가능하다. 변환 후 각 행에 대해 프롬프트 생성 및 토큰화를 적용하여 학습에 바로 활용 가능한 형태로 만든다.

6. 모델 정의

beomi/gemma-ko-2b 모델을 4bit 양자화(BitsAndBytesConfig) 방식으로 불러온다. 양자화는 모델의 파라미터를 더 적은 비트로 압축하여 메모리 사용량을 줄이는 기법으로, Colab 무료 GPU(T4, VRAM 15GB) 환경에서도 대형 언어모델을 실행할 수 있게 해준다. 이후 LoRA(LoraConfig)를 적용하여 전체 파라미터가 아닌 일부 핵심 레이어만 학습하도록 설정한다.

7. 학습

SFTTrainer를 사용하여 fine-tuning을 수행한다. 사전학습된 언어모델은 이미 방대한 텍스트를 학습한 상태이지만, 난독화 복원이라는 특수한 task에 최적화되어 있지 않다. fine-tuning을 통해 모델이 우리 데이터의 패턴을 학습하도록 하며, 이 단계가 없으면 모델은 난독화 복원을 제대로 수행하지 못한다.

8. 검증

학습이 완료된 모델을 val_split 데이터에 적용하여 문자 단위 F1 Score를 계산한다. 검증 단계는 모델이 학습 데이터에 과적합되지 않고 새로운 데이터에 대해서도 잘 작동하는지 확인하기 위해 반드시 필요하다. 이 점수를 기반으로 모델의 성능을 판단하고 개선 방향을 결정한다.

9. 테스트 추론

검증까지 완료된 모델을 test 데이터에 적용하여 난독화된 리뷰를 복원한다. test 데이터에는 정답(output)이 없으므로 성능 확인 없이 복원 결과만 생성한다.

10. 제출 파일 생성

sample_submission.csv의 형식에 맞춰 복원 결과를 output 컬럼에 채워 넣고 submission.csv로 저장한다. 제출 형식이 맞지 않으면 채점이 불가능하므로, ID와 output 컬럼 구조를 반드시 유지해야 한다.

3. Dictionary Baseline 설명

1) 문자열 치환 방식

```
1 match_dict = {}
2
3 # 단어단위로 사전 생성
4 # for input_text, output_text in zip(train_data['input'], train_data['output']):
5 #     input_words = input_text.split()
6 #     output_words = output_text.split()
7 #     for iw, ow in zip(input_words, output_words):
8 #         match_dict[iw] = ow
9
10 # 5.5. 오류분석에서 찾은 문제에 대한 해결방안 실험 - 음절 단위로 사전 만들어서 적용
11 for input_text, output_text in zip(train_data['input'], train_data['output']):
12     for iw, ow in zip(input_text, output_text):
13         match_dict[iw] = ow
```

입력 문장을 공백 기준으로 단어 단위로 나눈 뒤, 미리 구축한 사전(match_dict)에서 각 단어를 찾아 대응하는 원문 단어로 바꾸는 방식이다. 사전에 없는 단어는 원문 그대로 유지된다.

2) Dictionary 구성 방식

train 데이터의 input과 output 문장을 단어 단위로 나눈 뒤, zip으로 같은 위치 단어끼리 짝지어 {난독화 단어: 정답 단어} 형태로 저장하였다. train 데이터에서만 사전을 구축하였으며, val/test 데이터는 검증 및 예측에만 사용했다.

복원 적용

```
1 def replace_words(input_text, match_dict):
2     words = input_text.split()
3     replaced_words = [match_dict.get(word, word) for word in words]
4     return " ".join(replaced_words)
```

3) Exact matching 개념

사전에서 단어를 찾을 때 정확히 일치하는 단어만 매칭되는 방식이다. 한 글자라도 다르면 매칭에 실패하고, 그 단어는 치환되지 않은 채 그대로 출력된다.

위의 이미지 내 코드 3번 줄의 `match_dict.get(word, word)`에서 사전에 없는 단어는 원문 그대로 반환하는 것이 Exact Matching의 핵심이다.

4) Rule-based 방식의 한계

```
1 val_data['pred'] = val_pred
2 val_data['f1'] = [char_f1(p, a) for p, a in zip(val_pred, val_data['output'])]
3
4 worst_cases = val_data.sort_values('f1').head(20)
5
6 for _, row in worst_cases.iterrows():
7     print(f"입력: {row['input']}")
8     print(f"예측: {row['pred']}")
9     print(f"정답: {row['output']}")
10    print(f"F1: {row['f1']:.2f}")
11    print()
```

<실행결과 - 오류분석 사례>

입력: 워꾸첸꾸뽀뽀룸 쉼빔수 쿨치한넛옴.
예측: 워꾸첸꾸뽀뽀룸 쉼빔수 쿨치한넛옴.
정답: 이그제큐티브룸 서비스 끔찍하네옴.
F1: 0.17

입력: 캐꿀하콤 셉뽀쓰갸 콕끔윙탓.
예측: 캐꿀하콤 셉뽀쓰갸 콕끔윙탓.
정답: 깨끗하고 서비스가 고급이다.
F1: 0.20

...

입력: 권짼은테, 써뽀수는 웨약깃예옴.
예측: 권짼은테, 써뽀수는 웨약깃예옴.
정답: 괜찮은데, 서비스는 최악이에요.
F1: 0.29

입력: 양인툴괘 힐링학괘 종앙웃
예측: 양인툴괘 힐링학괘 종앙웃
정답: 아이들과 힐링하기 좋아요
F1: 0.30

입력: 깔끔하곤 야추양쭈 째 위웅헛쑤닌다 ㅎㅎ
예측: 깔끔하곤 야추양쭈 잘 위웅헛쑤닌다 ㅎㅎ
정답: 깔끔하고 아주아주 잘 이용했습니다 ㅎㅎ
F1: 0.30

train 데이터에서 본 적 없는 단어 조합은 전혀 복원하지 못한다. 오류 분석 결과, F1 점수가 낮은 사례들은 대부분 입력 문장이 예측 결과와 동일하게 출력되었는데, 이는 해당 언어가 train 사전에 정확히 일치하는 형태로 존재하지 않았기 때문이다.

단어 - Validation Char F1: 0.5684 음절 - Validation Char F1: 0.4191

```
1 from collections import Counter
2
3 def char_f1(pred, answer):
4     pred_chars = Counter(pred)
5     ans_chars = Counter(answer)
6
7     # 공통 문자 수 구하기
8     common = sum((pred_chars&ans_chars).values())
9
10    if common == 0:
11        return 0.0
12
13    # precision = TP / TP + FP
14    precision = common / sum(pred_chars.values())
15    # Recall = TP / TP + FN
16    recall = common / sum(ans_chars.values())
17
18    #F1 = 2 * (Precision * Recall) / (Precision + Recall)
19    return 2 * (precision * recall) / (precision + recall)
20
```

```
1 scores = [char_f1(pred, ans) for pred, ans in zip(val_pred, val_data['output'])]
2 print(f'Validation Char F1: {sum(scores)/len(scores):.4f}')
```

Validation Char F1: 0.4191

이 결과를 바탕으로 음절 단위로 바꾸는 실험을 진행하였으나, 오히려 F1 Score가 0.5684에서 0.4191로 하락하는 현상이 발생하였다. 아마 이미 있는 글자가 여러문장에 의하여 계속하여 다른 형태로 변형되다보니, 사전에 저장될때마다 값이 덮어써지면서 발생한 현상으로 보인다. 이는 매칭 단위를 쪼개는 것만으로는 일반화 성능이 개선되지 않고 오히려 문맥정보가 사라져 안정성이 떨어질 있음을 보여준다.

5) 새로운 표현에 취약한 이유

Rule-based 방식은 데이터를 “암기”하는 방식이지 “이해”하는 방식이 아니다. 문맥을 고려하는 것이 아닌 답을 외워서 정해진 패턴만 인식하고 대응하는 답을 출력하기 때문에, train데이터에 없는 새로운 조합이나 신조어에는 대응할 수 없다.

4. LLM Baseline 설명

1) 선택한 모델

본 과제에서는 총 세 가지 모델을 실험했다. 초기에는 바이트 단위 토큰나이징이 가능한 ByT5-Small을 사용했으나, 한국어 처리 비효율 및 낮은 성능(F1 0.298)으로 인해 한국어 특화 모델인 KoBART로 전환했다. 최종적으로 데이터 증강 실험에서는 beomi/gemma-ko-2b를 사용했다.

2) Tokenizer 구조

Tokenizer는 텍스트를 모델이 처리할 수 있는 숫자(토큰) 단위로 변환하는 역할을 한다. ByT5는 바이트 단위, KoBART는 한국어 형태소 기반 BPE(Byte Pair Encoding) 방식을 사용한다. 형태소 기반 토큰나이저는 한국어의 교착어적 특성을 더 잘 반영하므로, 동일한 문장 길이에서 더 의미 있는 단위로 토큰화가 가능하다. 이로 인해 KoBART가 ByT5 대비 훨씬 빠른 수렴 속도를 보였다.

3) Encoder-Decoder 구조

ByT5, KoBART 모두 Encoder-Decoder 구조를 기반으로 한다. Encoder는 입력 문장(난독화된 텍스트)의 의미를 압축해 내부 표현으로 변환하고, Decoder는 이 표현을 바탕으로 복원된 문장을 순차적으로 생성한다. 이 구조는 입력과 출력의 형태가 다른 seq2seq 과제(번역, 복원 등)에 적합하다. 반면 gemma-ko-2b는 Decoder-Only 구조로, 프롬프트(입력)를 받아 이어지는 텍스트를 생성하는 방식으로 동작한다.

4) Pretrained의 의미

사전학습(pretrained)이란 대규모 텍스트 데이터로 모델을 미리 학습시켜 언어의 문법, 문맥, 의미 관계를 이미 파악한 상태를 말한다. KoBART는 한국어 뉴스, 위키, 책 등 대량의 한국어 데이터로 사전학습되어 있어, 한국어 문맥 파악 능력이 이미 내재되어 있다. 이 덕분에 소량의 task-specific 데이터(난독화 리뷰)만으로도 빠르게 높은 성능을 달성할 수 있다.

5) Fine-tuning 방식

사전학습된 모델을 난독화 복원 task에 맞게 추가 학습하는 과정이다. gemma-ko-2b 실험에서는 전체 파라미터를 학습시키는 대신, LoRA(Low-Rank Adaptation) 기법을 적용해 일부 핵심 레이어(q_proj, v_proj)만 학습했다. LoRA는 학습 파라미터 수를 대폭 줄여 제한된 GPU 자원(Colab T4)에서도 대형 모델을 fine-tuning할 수 있게 해준다. 또한 4bit 양자화를 적용해 메모리 사용량을 추가로 줄였다.

6) Generation 과정

fine-tuning된 모델에 난독화된 입력 문장을 프롬프트 형태로 넣으면, 모델이 복원된 문장을 순차적으로 생성한다. ByT5, KoBART는 Beam Search(빔 탐색) 방식으로, gemma-ko-2b는 Greedy Decoding 또는 Beam Search 방식으로 출력을 생성했다. Beam Search는 여러 후보 경로를 동시에 탐색해 더 자연스러운 문장을 생성하는 방식이다.

5. 실험 결과 비교 → 실험 실패...

(1) 모델 구조 비교 실험

본 실험에서는 초기 모델로 ByT5-Small을 선정하여 베이스라인을 구축하였으나, 한국어 처리에 있어 심각한 구조적 비효율성을 발견하였다. 처음에는 ByT5가 바이트 단위로 토큰나이즈를 하기 때문에, 음절 단위까지 복원할 수 있어서 성능이 좋을 것이라고 판단했다.

그러나 ByT5는 바이트(Byte) 단위의 범용적인 토큰화 방식을 채택하고 있어, 256 토큰 길이의 시퀀스 처리 시 학습에만 5시간 이상의 과도한 연산 시간이 소요되었다. 또한, 한국어의 교착어적 특성과 문맥적 구조를 바이트 단위에서 재구성하는 데 한계가 있어, 최종 Char F1 스코어가 0.298 수준에서 정체되는 현상을 보였다.

이에 아키텍처 피벗을 단행하여, 한국어 형태소 분석과 문맥 파악에 특화된 KoBART 기반의 모델로 전환하였다. 해당 아키텍처로 변경한 실험(Exp 2)에서는 동일한 컴퓨팅 자원 내에서 손실값(Loss)이 0.099에서 0.052로 약 47%가량 급격히 감소하며 훨씬 우수한 수렴 속도를 보였다. 이를 통해 한국어 NLP 작업에서는 무작정 범용 모델을 사용하는 것보다, 해당 언어의 언어학적 특성을 사전에 학습한 모델을 사용하는 것이 자원 효율성과 성능 측면에서 압도적으로 유리함을 확인하였다.

Exp1.

```
[ ] # 1. 필수 라이브러리 설치
!pip install -q transformers datasets accelerate evaluate

# 2. 구글 드라이브 연동
from google.colab import drive
drive.mount('/content/drive')

... Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

[ ] import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from datasets import Dataset
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
from transformers import DataCollatorForSeq2Seq, Seq2SeqTrainingArguments, Seq2SeqTrainer

import random
from tqdm.notebook import tqdm
tqdm.pandas()

[ ] # =====
# 1. 데이터 로드
# =====

data_path = '/content/drive/MyDrive/시냅스 팀물_2/dataset/train.csv'
df = pd.read_csv(data_path)
train_df, val_df = train_test_split(df, test_size=0.1, random_state=42)

train_dataset = Dataset.from_pandas(train_df)
val_dataset = Dataset.from_pandas(val_df)

[ ] from google.colab import drive
drive.mount('/content/drive')

... Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

[ ] # =====
# 2. ByT5 모델 및 토큰라이저 로드
# =====

model_name = "google/byt5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

Loading weights: 100% 172/172 [00:00<00:00, 2591.276/s]
[transformers] The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are present in the checkpoints with different values, so we will NOT tie them.

[ ] # =====
# 3. 데이터 전처리 (도론화)
# =====

def preprocess_function(examples):
    inputs = examples["input"]
    targets = examples["output"]

    # 입력과 정답 모두 max_length=256으로 채워서 고정
    model_inputs = tokenizer(inputs, max_length=256, padding="max_length", truncation=True)
    labels = tokenizer(targets, max_length=256, padding="max_length", truncation=True)

    # * 핵심: 정답(label)의 빈칸(pad) 토큰을 -100으로 바꿔서 모델이 계산할 때 무시하도록 강제
    labels_with_ignore_index = []
    for label in labels["input_ids"]:
        # 패딩 토큰이면 -100으로, 아니면 원래 토큰 유지
        label = [l if l != tokenizer.pad_token_id else -100 for l in label]
        labels_with_ignore_index.append(label)

    model_inputs["labels"] = labels_with_ignore_index
    return model_inputs
```

<pre> 11 # ===== 12 # 4. 평가 지표 (Char F1 Score) 계산 함수 13 # ===== 14 def calc_char_f1(pred_str, true_str): 15 pred_chars = list(pred_str.replace(" ", "")) 16 true_chars = list(true_str.replace(" ", "")) 17 18 if len(pred_chars) == 0 or len(true_chars) == 0: 19 return 0.0 20 21 common_chars = set(pred_chars) & set(true_chars) 22 if not common_chars: 23 return 0.0 24 25 precision = len(common_chars) / len(set(pred_chars)) 26 recall = len(common_chars) / len(set(true_chars)) 27 28 f1 = 2 * (precision * recall) / (precision + recall) 29 return f1 30 31 def compute_metrics(eval_preds): 32 preds, labels = eval_preds 33 decoded_preds = tokenizer.batch_decode(preds, skip_special_tokens=True) 34 labels = np.where(labels != -100, labels, tokenizer.pad_token_id) 35 decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True) 36 37 f1_scores = [calc_char_f1(p, l) for p, l in zip(decoded_preds, decoded_labels)] 38 return {"char_f1": np.mean(f1_scores)} </pre>	
<pre> 11 # ===== 12 # 5. 모델 학습 설정 (드라이브에 저장되도록 수정) 13 # ===== 14 15 def preprocess_function(examples): 16 inputs = examples["input"] 17 targets = examples["output"] 18 19 # 변경점: padding="max_length" 옵션을 추가하여 attention_mask가 정해지지 않게 고정합니다. 20 model_inputs = tokenizer(inputs, max_length=256, padding="max_length", truncation=True) 21 labels = tokenizer(targets, max_length=256, padding="max_length", truncation=True) 22 23 model_inputs["labels"] = labels["input_ids"] 24 return model_inputs </pre>	
<pre> 11 from transformers import DataCollatorForSeq2Seq 12 import numpy as np 13 14 # ===== 15 # 1. 필수 부분 16 # ===== 17 data_collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, model=model) 18 19 def calc_char_f1(pred_str, true_str): 20 pred_chars = list(pred_str.replace(" ", "")) 21 true_chars = list(true_str.replace(" ", "")) 22 23 if len(pred_chars) == 0 or len(true_chars) == 0: return 0.0 24 common_chars = set(pred_chars) & set(true_chars) 25 if not common_chars: return 0.0 26 precision = len(common_chars) / len(set(pred_chars)) 27 recall = len(common_chars) / len(set(true_chars)) 28 return 2 * (precision * recall) / (precision + recall) 29 30 def compute_metrics(eval_preds): 31 preds, labels = eval_preds 32 decoded_preds = tokenizer.batch_decode(preds, skip_special_tokens=True) 33 labels = np.where(labels != -100, labels, tokenizer.pad_token_id) 34 decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True) 35 f1_scores = [calc_char_f1(p, l) for p, l in zip(decoded_preds, decoded_labels)] 36 return {"char_f1": np.mean(f1_scores)} </pre>	

```

# =====
# 2. 모델 세팅
# =====
training_args = Seq2SeqTrainingArguments(
    output_dir="/results",
    eval_strategy="epoch",
    learning_rate=3e-4,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    weight_decay=0.01,
    save_total_limit=2,
    num_train_epochs=5,
    predict_with_generate=True,
    fp16=False,
    logging_steps=50,
    report_to="none"
)

trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_val,
    data_collator=data_collator,
    compute_metrics=compute_metrics
)

# =====
# 3. 학습 시작
# =====
print("드디어 에러를 잡았습니다. 학습을 시작합니다!!!")

# 1. 이상한 토큰 생성 방지를 위한 데이터 필터링
def filter_bad_tokens(example):
    # 라벨이 >114112 이상인 값이 들어가는 것을 방지하기 위한 안전장치
    return all((label == -100 or 0 <= label < 114112 for label in example["labels"]))

print("데이터셋 안전 검사 중...")
tokenized_train = tokenized_train.filter(filter_bad_tokens)
tokenized_val = tokenized_val.filter(filter_bad_tokens)

# 2. 학습 시작
trainer.train()

```



Exp2.

```
[ ] # =====
# 1. 필수 라이브러리 설치
# =====
!pip install -q transformers datasets accelerate evaluate

84.1/84.1 kB 2.6 MB/s eta 0:00:00

[ ] import torch
print("GPU 사용 가능:", torch.cuda.is_available())
print("GPU 이름:", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "없음")

GPU 사용 가능: True
GPU 이름: Tesla T4

[ ] # =====
# 2. 구글 드라이브 연동
# =====
from google.colab import drive
drive.mount('/content/drive')

import pandas as pd
import numpy as np
from collections import Counter
from sklearn.model_selection import train_test_split
from datasets import Dataset
from transformers import (
    PreTrainedTokenizerFast,
    BartForConditionalGeneration,
    DataCollatorForSeq2Seq,
    Seq2SeqTrainingArguments,
    Seq2SeqTrainer
)

... Mounted at /content/drive

[ ] # =====
# 3. 데이터 로드
# =====
data_path = '/content/drive/MyDrive/시놉스 퓌물_2/dataset/train.csv'
df = pd.read_csv(data_path)

train_df, val_df = train_test_split(df, test_size=0.1, random_state=42)
train_dataset = Dataset.from_pandas(train_df, preserve_index=False)
val_dataset = Dataset.from_pandas(val_df, preserve_index=False)

print(f"학습 데이터: {len(train_dataset)}개 | 검증 데이터: {len(val_dataset)}개")

학습 데이터: 10136개 | 검증 데이터: 1127개
```

[]	<pre> # ===== # 4. KoBART 모델 & 토큰라이저 로드 # ===== MODEL_NAME = "gogamza/kobart-base-v2" tokenizer = PreTrainedTokenizerFast.from_pretrained(MODEL_NAME) model = BartForConditionalGeneration.from_pretrained(MODEL_NAME) print("KoBART 로드 완료!") </pre>
v	<pre> /usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:112: UserWarning: The secret `HF_TOKEN` does not exist in your Colab secrets. To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session. You will be able to reuse this secret in all of your notebooks. Please note that authentication is recommended but still optional to access public models or datasets. warnings.warn(WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads. WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads. tokenizer.json: 100% ██████████ 682k/682k [00:00<00:00, 21.0MB/s] added_tokens.json: 100% ██████████ 4.00/4.00 [00:00<00:00, 429B/s] special_tokens_map.json: 100% ██████████ 112/112 [00:00<00:00, 11.0kB/s] config.json: 100% ██████████ 1.36k/1.36k [00:00<00:00, 129kB/s] model.safetensors: 100% ██████████ 495M/495M [00:04<00:00, 175MB/s] Loading weights: 100% ██████████ 259/259 [00:00<00:00, 5641.78kB/s] KoBART 로드 완료! </pre>
[]	<pre> # ===== # 5. 토큰라이징 # ===== MAX_LENGTH = 128 # 256은 너무 길어서 느림 - 128로 충분 def preprocess_function(examples): model_inputs = tokenizer(examples["input"], max_length=MAX_LENGTH, truncation=True, padding=False, # - padding은 DataCollator가 담당) labels = tokenizer(text_target=examples["output"], max_length=MAX_LENGTH, truncation=True, padding=False,) model_inputs["labels"] = labels["input_ids"] return model_inputs </pre>
[]	<pre> tokenized_train = train_dataset.map(preprocess_function, batched=True, remove_columns=train_dataset.column_names) tokenized_val = val_dataset.map(preprocess_function, batched=True, remove_columns=val_dataset.column_names) print("토큰라이징 완료!") </pre>
v	<pre> Map: 100% ██████████ 10136/10136 [00:02<00:00, 4664.80 examples/s] Map: 100% ██████████ 1127/1127 [00:00<00:00, 4514.48 examples/s] 토큰라이징 완료! </pre>
[]	<pre> # ===== # 6. F1 계산 (Counter 버전 - 버그 수정) # ===== def calc_char_f1(pred_str, true_str): pred_chars = list(pred_str.replace(" ", "")) true_chars = list(true_str.replace(" ", "")) if len(pred_chars) == 0 and len(true_chars) == 0: return 1.0 if len(pred_chars) == 0 or len(true_chars) == 0: return 0.0 pred_counter = Counter(pred_chars) true_counter = Counter(true_chars) common = sum((pred_counter & true_counter).values()) # 중복 포함 교집합 precision = common / len(pred_chars) recall = common / len(true_chars) if precision + recall == 0: return 0.0 return 2 * precision * recall / (precision + recall) def compute_metrics(eval_preds): preds, labels = eval_preds decoded_preds = tokenizer.batch_decode(preds, skip_special_tokens=True) labels = np.where(labels != -100, labels, tokenizer.pad_token_id) decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True) f1_scores = [calc_char_f1(p, l) for p, l in zip(decoded_preds, decoded_labels)] return {"char_f1": np.mean(f1_scores)} </pre>

```
[ ] # =====
# 7. 학습 세팅
# =====
data_collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, model=model)

training_args = Seq2SeqTrainingArguments(
    output_dir="./results_kobart",
    eval_strategy="epoch",
    learning_rate=3e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    weight_decay=0.01,
    warmup_steps=100,
    save_total_limit=2,
    num_train_epochs=10,
    predict_with_generate=True,
    fp16=True,
    logging_steps=50,
    report_to="none",
    generation_max_length=128,
)

trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_val,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)

[ ] # =====
# 8. 학습 시작
# =====
print("KoBART 학습 시작!")
trainer.train()

KoBART 학습 시작! [ 837/6340 03:53 < 25:36, 3.58 R/s, Epoch 1.32/10]
Epoch Training Loss Validation Loss Char F1
1 0.569385 0.885300 0.433314
Writing model shards: 100% 1/1 [00:01<00:00, 1.68s/s]
```

**Epoch 3번 돌려서 .4 후반대까지 오른 걸 확인했는데 GPU 터지고 당황해서 초기화했다가 이렇게 됨

(2) 데이터 증강 실험

데이터 증강의 강건성을 검증하기 위해 원본 데이터에 두 가지 노이즈 기법을 적용할 계획을 수립하였다. 첫째는 한국어 디지털 텍스트에서 빈번한 띄어쓰기 변조(공백 50% 제거)이며, 둘째는 'ㄱ', 'ㅎ' 등 구어체 특수 기호를 무작위 삽입하여 모델의 노이즈 강건성(Noise Robustness)을 테스트하고자 하였다. 실험(Exp 3)을 통해 데이터의 양을 2배로 확장하고 이러한 노이즈를 학습시켜, 실전 데이터 환경에서도 모델이 안정적인 추론을 수행할 수 있도록 일반화 성능을 유도하였다.

그러나 해당 실험 과정에서 대규모 LLM 파인튜닝으로 인한 구글 코랩의 T4 GPU 할당량 초과 문제가 발생하였다. 이를 해결하기 위해 중단된 학습 시점의 모델 가중치를 구글 드라이브에 백업하였으며, CPU 환경에서 샘플 데이터를 활용한 학습 파이프라인 검증을 하였으나, GPU를 사용할 수 없어 고착된 상태로 실험을 종료하게 되었다. 향후 GPU 자원 복구 시 중단된 지점부터 학습을 재개하여 증강된 모델의 최종 F1 스코어를 산출하고자 한다.

```

111 # =====
# 데이터 중앙 실행 (baseline\_lm.ipynb 기반)
# =====
# * 위에서 baseline 코드 (모델 로드, create\_prompt, char\_f1 등) 실행 후 여기서 실행

import random
import pandas as pd
from datasets import Dataset, concatenate\_datasets
from collections import Counter

[14]
✓ 2%

# 1. 구글 드라이브 마운트 (혹시 재부팅 되어서 끊겼을까봐 다시 확인)
from google.colab import drive
drive.mount('/content/drive')

# 2. 과제 데이터 파일 경로 복사해서 붙여넣기 ✨
file\_path = '/content/drive/MyDrive/시놉스 토플\_2/dataset/train.csv'

# 3. 데이터를 train\_data라는 이름으로 불러오기 * CPU용으로 100개만 불러기
# 만약 엑셀 파일(.xlsx)이라면 pd.read\_excel(file\_path)로 바꾸셔야 합니다!
train\_data = pd.read\_csv(file\_path)
train\_data = train\_data.head(100) # CPU에서 3초 만에 끝나도록 100개만 샘플링!

print("데이터 연결 완벽 성공! 데이터 개수:", len(train\_data))

✓
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force\_remount=True).
데이터 연결 완벽 성공! 데이터 개수: 100

[15]
✓ 0%

# =====
# 1. 중강 함수 정의
# =====
CHOSUNG\_LIST = ['ㄱ','ㄴ','ㄷ','ㄹ','ㅁ','ㅂ','ㅅ','ㅇ','ㅈ','ㅊ','ㅋ','ㆁ','ㅅ','ㅈ','ㅊ','ㅋ','ㆁ']

def get\_chosung(char):
    if '가' <= char <= '힣':
        return CHOSUNG\_LIST[(ord(char) - 0xA000) // (21 * 28)]
    return char

# 방식 1: 랜덤 픽어쓰기 삽입 - "중어요" -> "중 아 요"
def aug\_random\_spacing(text, prob=0.3):
    result = []
    for char in text:
        result.append(char)
        if char != ' ' and random.random() < prob:
            result.append(' ')
    return ''.join(result).strip()

----
✓ 0%

# 방식 2: 초성 변환 - "최고" -> "조고"
def aug\_chosung(text, prob=0.3):
    return ''.join(
        get\_chosung(c) if '가' <= c <= '힣' and random.random() < prob else c
        for c in text
    )

# 방식 3: 유사 발음 치환 - "중다" -> "조다"
PHONETIC\_MAP = {
    '중': '조', '일': '일', '일': '일', '일': '일', '일': '일',
    '말': '말', '말': '말', '말': '말', '말': '말', '말': '말',
    '새': '새', '의': '의', '의': '의', '의': '의', '의': '의',
    '하': '하', '하': '하', '하': '하', '하': '하', '하': '하',
    '했': '했', '했': '했', '했': '했', '했': '했',
}

def aug\_phonetic(text, prob=0.4):
    return ''.join(
        PHONETIC\_MAP[c] if c in PHONETIC\_MAP and random.random() < prob else c
        for c in text
    )

# 방식 4: 특수문자 삽입 - "중어요" -> "중어요"
SPECIAL\_CHARS = ['<','>','@','^','!','?','_','_',';']

def aug\_special\_char(text, prob=0.15):
    result = []
    for char in text:
        result.append(char)
        if char != ' ' and random.random() < prob:
            result.append(random.choice(SPECIAL\_CHARS))
    return ''.join(result)

# 방식 5: 반복 문자 추가 - "중어요"
REPEAT\_CHARS = ['<','>','@','^','!','?','_','_',';']

def aug\_repeat\_char(text, prob=0.15):
    result = []
    for char in text:
        result.append(char)
        if '가' <= char <= '힣' and random.random() < prob:
            result.append(random.choice(REPEAT\_CHARS) * random.randint(1, 3))
    return ''.join(result)

ALL\_METHODS = [aug\_random\_spacing, aug\_chosung, aug\_phonetic, aug\_special\_char, aug\_repeat\_char]

```

151 ✓ 0.8	<pre>def augment_text(text): selected = random.sample(ALL_METHODS, k=random.randint(1, 3)) result = text for method in selected: result = method(result) return result</pre>	
161 ✓ 0.8	<pre># ===== # 2. 증강 데이터셋 생성 # ===== def create_augmented_df(df, aug_multiplier=2): """원문(output)에 난독화를 새로 적용해서 (새 input, 동일 output) 쌍 생성""" rows = [] for _, row in df.iterrows(): original = row['output'] for _ in range(aug_multiplier): rows.append({ 'input': augment_text(original), 'output': original }) return pd.DataFrame(rows) print("데이터 증강 중...") aug_df = create_augmented_df(train_data, aug_multiplier=2) combined_df = pd.concat([train_data, aug_df], ignore_index=True).sample(frac=1, random_state=42) print(f"원본: {len(train_data):,}개") print(f"증강: {len(aug_df):,}개") print(f"합계: {len(combined_df):,}개") # 샘플 확인 print("\n[증강 샘플 3개]") for i in range(3): print(f" 원문: {aug_df.iloc[i]['output']}") print(f" 난독화: {aug_df.iloc[i]['input']}") print()</pre>	
✓	데이터 증강 중... 원본: 100개 증강: 200개 합계: 300개 [증강 샘플 3개] 원문: 별 한 개도 아랍다. 왜 사람들이 별 1개를 주는지 적어본 사람으로서 말로 설명하하니 댓글로는 너무 길고... 아무튼 두 번 다시 가질 싶은 곳. 여행을 20여 년 다녀본 곳 중 제일 기분 나빴던 곳. 난독화: 별 한 개도 아랍다. 왜 사 람 들 이 별 1 개를 주 는지 적어본 사람우 로서 말로 설명하이니 댓글로는 너 무 길 고. . . 아무튼 두 번 다시 가 질 싶 은 곳 . 캠 팅을 2 0여 년 다 다녀본 곳 중 제 일 기 분 나빴던 곳 . 원문: 별 한 개도 아랍다. 왜 사람들이 별 1개를 주는지 적어본 사람으로서 말로 설명하하니 댓글로는 너무 길고... 아무튼 두 번 다시 가질 싶은 곳. 여행을 20여 년 다녀본 곳 중 제일 기분 나빴던 곳. 난독화: 별 한 개도 아랍다. 왜^ 사람 들 이 별 1개를 주 는지ㅠ 꺾^ 어본 사렘ㅠ로서디 말로 설명 하하니 댓글로?는 너무 길고... 아예무튼 두 번 다시 가질!!음!# 싶은 곳ㅠ . 캅^ 팅?을!ㅠ!! 2 0 여 년!#! 다-나려본 곳 중 제 일 기 분 나빴던 곳. 원문: 장만 자고 갈 때 좋네요. 장수도 중 ㅋㅋ 난독화: x만 x고 갈 때 좋네요. 줘수도 x ㅋㅋ	
1241 ✓ 0.8	<pre>from transformers import AutoTokenizer # 읽어버린 Gemma 토큰라이저 기억 강제 주입! # (주의: baseline 코드에 적혀있던 정확한 gemma 모델명이 있다면 아래 따옴표 안을 그대로 바꿔주세요!) model_id = "beomi/gemma-ko-2b" tokenizer = AutoTokenizer.from_pretrained(model_id) # ===== # 3. 증강 데이터를 baseline 포맷으로 변환 # ===== aug_train_dataset = Dataset.from_pandas(combined_df, preserve_index=False) def format_chat_template(row): prompt = create_prompt(row['input'], row['output']) # baseline의 함수 재사용 tokens = tokenizer.encode(prompt, truncation=True, max_length=256) row['input_ids'] = tokens return row aug_train_dataset = aug_train_dataset.map(format_chat_template, batched=False) print("✓ 증강 데이터 토큰라이징 완료")</pre>	
✓	/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:112: UserWarning: The secret `HF_TOKEN` does not exist in your Colab secrets. To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session. You will be able to reuse this secret in all of your notebooks. Please note that authentication is recommended but still optional to access public models or datasets. warnings.warn(config.json: 100% ██████████ 645/645 [00:00<00:00 34.2kB/s] Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads. WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads. tokenizer.config.json: 100% ██████████ 111k/111k [00:00<00:00 49.3kB/s] tokenizer.json: 100% ██████████ 17.5M/17.5M [00:00<00:00 29.4MB/s] special_tokens_map.json: 100% ██████████ 555/555 [00:00<00:00 39.2kB/s] Map: 100% ██████████ 300/300 [00:00<00:00 2356.93 examples/s] ✓ 증강 데이터 토큰라이징 완료	
1251 ✓ 0.8	<pre># ===== # 3. 증강 데이터를 baseline 포맷으로 변환 # ===== aug_train_dataset = Dataset.from_pandas(combined_df, preserve_index=False) def format_chat_template(row): prompt = create_prompt(row['input'], row['output']) # baseline의 함수 재사용 tokens = tokenizer.encode(prompt, truncation=True, max_length=256) row['input_ids'] = tokens return row</pre>	
1251 ✓ 0.8	aug_train_dataset = aug_train_dataset.map(format_chat_template, batched=False) print("✓ 증강 데이터 토큰라이징 완료")	
✓	Map: 100% ██████████ 300/300 [00:00<00:00 1957.36 examples/s] ✓ 증강 데이터 토큰라이징 완료	
1271 ✓ 0.8	<pre>!pip install -q trl peft</pre>	
✓	825.1/825.1 kB 30.4 MB/s eta 0:00:00 555.1/555.1 kB 28.5 MB/s eta 0:00:00 48.9/48.9 MB 11.3 MB/s eta 0:00:00	

**여기까지 하고 GPU 동시에 터짐 → 끝...

실험 번호	모델	Pretrai ned	Aug	Decoding	LR	Batch Size	F1
Exp 1	ByT5-Small	Yes	None	Beam	5e-5	16	0.2987*
Exp 2	KoBART	Yes	None	Greedy	3e-5	8	0.4333**
Exp 3	Gemma-ko -2B	Yes	Yes	Beam	2e-5	4	TBD***

*오류 확인 후 epoch 3 경과 후 실험 종료

**GPU 자원 제한으로 일시 중단

***GPU 자원 제한으로 일시 중단

6. 결과 해석

1) Dictionary 방식과 LLM 방식의 차이는 무엇인가?

Dictionary 방식은 train 데이터에서 추출한 단어 쌍 사전을 기반으로 정확히 일치하는 단어만 치환하는 rule-based 방식이다. 구현이 단순하고 속도가 빠르지만, 사전에 없는 새로운 난독화 표현에는 전혀 대응하지 못한다는 한계가 있다. 반면 LLM 방식은 사전학습된 언어 모델이 문맥을 파악하여 복원 결과를 생성하므로, 처음 보는 표현에도 추론이 가능하다. 다만 학습 시간, GPU 자원, 복잡한 환경 설정이 필요하다는 단점이 있다.

2) 어떤 모델의 성능이 가장 높았는가?

완료된 실험 중에서는 KoBART 기반 Exp 2가 Char F1 Score 0.4333으로 가장 높은 성능을 보였다. ByT5-Small 기반 Exp 1은 0.2987로 낮았으며, gemma-ko-2b 기반 Exp 3은 GPU 자원 한계로 완료되지 못해 정량적 비교가 불가능했다. 한국어 특화 사전학습 모델(KoBART)이 범용 모델(ByT5)보다 유의미하게 높은 성능을 보임을 확인했다.

3) Augmentation은 어떤 영향을 주었는가?

데이터 증강 실험(Exp 3)은 GPU 자원 한계로 완료되지 못해 정량적 비교는 불가능했다. 다만 증강으로 인해 학습 데이터가 2배(약 22,526개)로 늘어나며 학습 스텝 수도 2배가 되었고, 이로 인해 Colab 무료 GPU 환경에서 학습 완료 자체가 어려워졌다. 향후 GPU 자원이 확보된다면, 증강 데이터를 통해 특수문자 삽입 등 다양한 난독화 패턴에 대한 모델의 강건성이 높아질 것으로 예상된다.

4) Pretrained 모델은 왜 성능이 높은가?

사전학습 모델은 대규모 텍스트를 통해 언어의 문법, 문맥, 의미 관계를 이미 내재화하고 있다. 이 덕분에 난독화 복원이라는 특수한 task에 대해 소량의 데이터만으로 fine-tuning하더라도 빠르게 높은 성능에 도달할 수 있다. 반면 사전학습 없이 처음부터 학습(from scratch)하면 훨씬 많은 데이터와 시간이 필요하다. KoBART가 ByT5 대비 빠른 수렴과 높은 성능을 보인 것도, 한국어 특화 사전학습의 효과로 볼 수 있다.

5) 어떤 난독화 패턴이 가장 어려웠는가?

음절 단위로 완전히 뒤섞인 경우(예: '이그제큐티브룸' → '익귀츨뎡뵐뵐룸')가 가장 어려운 패턴으로 확인되었다. Dictionary 방식의 오류 분석에서 이 유형의 F1 Score가 0.17~0.24 수준으로 가장 낮았다. 단어 전체가 완전히 새로운 조합으로 변형된 경우, 사전 기반 매칭은 물론 LLM도 정확한 복원이 어렵다.

6) 추가 성능 개선 방법은 무엇인가?

첫째, 더 많은 에폭(epoch)으로 학습하거나, KoBART 기반에서 추가 하이퍼파라미터 튜닝을 진행하면 성능이 더 향상될 것으로 예상된다 (Exp 2에서 epoch 3 기준 F1 0.4 후반대까지 확인된 바 있음). 둘째, 데이터 증강 실험을 완료하면 모델이 다양한 난독화 패턴에 더 강건해질 것으로 기대된다. 셋째, Dictionary 방식과 LLM 방식을 결합(양상블)하는 방법도 고려할 수 있다. 사전에 있는 단어는 Dictionary로 처리하고, 없는 단어는 LLM이 추론하는 하이브리드 방식이 두 방법의 장점을 모두 활용할 수 있다.

7. 팀원별 기여도

이름	담당 역할	실제 수행 내용	이해한 핵심 개념
신윤서	팀원 A	- train/test 데이터 로드 및 기본 통계 확인 - 결측치 검토 - 문장 길이 분포 분석 및 시각화 - 문자 단위 난독화 패턴 분석 및 빈도 시각화 - Validation Split 수행	- 난독화 패턴의 구조적 특성 (모음 교체 중심) - train/test 데이터의 구조적 차이
박수빈	팀원 B	- baseline_dictionary.ipynb 실행 및 분석 - 문자 단위 F1 Score 함수 구현 - validation 결과 기반 오류 분석 수행 - 단어 단위 vs 음절 단위 사전 비교 실험 진행	- zip()을 통한 데이터 짝짓기 - exact matching 기반 사전 치환의 원리와 한계 - 문자 단위 F1 Score(Precision /Recall) 계산 방식 - rule-based 모델이 일반화에 취약한 이유
박소현	팀원 C	- 성능 개선 실험 진행 (모델 구조 비교 실험; ByT5, KoBART, 데이터 증강 실험)	- 성능 개선 실험 진행 시 필요한 자원 및 실험 실패에 대한 고찰

8. 생성형 AI 사용 기록

박수빈

: 코드의 전체흐름을 이해하기 위한 질문 및 zip() 함수 이해, tr 라이브러리 버전 차이로 인한 SFTTrainer 에러 발생 해결, F1 score 계산에 관련하여 질문하고 코드 작성 방법에 대해 도움을 받았다.

zip함수의 경우 동작원리가 헛갈려 예시를 보면서 이해하였다. SFTTrainer에러는 발생한 에러를 에러 발생한 코드를 주석처리하거나 수정하면서 해결해보려고 하였지만 실패하여 도움을 받아 수정하였다(Training Arguments 대신 SFTConfig 사용). 마지막으로 F1 Score은 이미 있는 코드를 사용해본경험밖에 없어 도움을 받아서 기본 코드를 작성하였고, 작성한 코드를 바탕으로 음절단위로 바꾸어 실험할 때에는 목적에 맞도록 F1 score 직접 수정하였다.

신윤서

: 난독화 패턴 분석 방법에 대해 질문하고 관련 코드 작성 방법을 질문하였다. 특히 문자 단위로 입력과 출력을 비교하여 어떤 문자가 어떤 문자로 변환되는지 추출하는 방법, 한글 폰트 적용 방법에 대해 도움 받았다.

Colab에서 한글 폰트가 깨지는 문제가 발생하였다. 처음에는 apt-get으로 나눔 폰트를 설치하는 방식을 시도하였으나 맥북 환경에서는 런타임 재시작 후에도 적용이 되지 않았다. 이후 GitHub에서 폰트 파일을 직접 다운로드하여 fontManager.addfont()로 등록하는 방식으로 해결하였다.

박소현

솔직히 말하자면 전공 학회 발표 이슈 & 아직 3과목 종강을 못해서(22일 자정에 모든 과목 종강...) AI에게 외주를 맡기다시피 했는데 그 업보를 받은 것 같다.